Can a Marketplace Spot Late Deliveries on the Day They Are Ordered?

Purchase-time delay triage under a daily review budget on the Olist marketplace

IS5126 Final Project · Group: *[group number and members]* · Evidence: formal run `experiment-v2-002`, second-period run `experiment-v3-late2017-001`, analysis layers in `artifacts/analysis/`

Executive summary

Late deliveries are the single clearest driver of bad reviews on Olist. Among 96,470 delivered orders, 6.8% arrived after the promised date, and 52.5% of those late orders received a one-star review, against 6.6% of on-time orders (mean score 2.27 versus 4.29).

We studied one concrete operating decision. **Every evening, an operations team can review at most 10% of the orders placed that day. Which orders should it pick, using only what was known when each order was placed, and is acting at that moment worth it?**

We compared 29 ways of ranking orders: 5 simple rules and 24 machine-learning models (logistic regressions and shallow decision trees). All were evaluated in time order. The winner was chosen on four monthly validation folds (January to April 2018) and then tested once on a later 96-day window (26 May to 29 August 2018). To check whether the conclusions survive a different period, we then froze a second design with three new rules and censoring-aware labels, and tested it on late 2017 (selection on June to September, test on 26 October to 31 December), a period on which no model had been scored before.

What we found

1. **A simple rule won, and machine learning added nothing.** The selected rule multiplies the historical late rate of the order's seller-state to customer-state route by how tight the promised delivery window is. It caught 25.2% of late orders in validation, against 21.0% for the best model. No model beat it in the backtest either.
2. **The signal at purchase time is weak.** In the backtest the rule flagged 1,885 orders and caught 117 of the 674 that turned out late. That is 17.4% recall and 6.2% precision, against a 3.5% base rate. The team would review about 16 orders for every late order found.
3. **What makes an order late changed between periods.** In early 2018, delays were concentrated on cross-state routes (14.2% late, versus 4.9% within a state). In the backtest window, short promises on local orders became the problem. Orders promised within 10 days made up 15.0% of orders, up from 1.2% earlier, and were late 11.6% of the time. São Paulo to São Paulo orders with such short promises alone accounted for 41% of all late orders. A rule that was useless in validation (ranking by shortest promise) did best in the backtest. Its edge is not statistically clear, and we did not switch to it after seeing the test.
4. **The second, pre-registered test confirms the main conclusions and improves the rule.**
    - Models again did not beat the rules.
    - The new rule that uses only the last 60 days of route history won selection and caught 21.1% of late orders (21.6% precision, 10.0% base rate). That is 7.7 percentage points more than the original rule, with a paired interval of 4.5 pp to 12.8 pp.

    - The two rules that looked best in hindsight on 2018 failed: promise slack caught 8.3% and shortest promise 4.2%, below a random queue.
5. **Whether acting at purchase time pays depends on how many orders are late.** Under the three cost scenarios fixed in advance, the purchase-time queue loses money in the low-delay 2018 window except in the most optimistic scenario, but pays in the moderate scenario in the high-delay late-2017 window.
6. **A later and cheaper signal is sharper in both periods.** Flagging orders that the seller has not handed to the carrier by the shipping-limit deadline reached 19.4% precision and 45.4% recall in 2018, where we found it after seeing the results. On the untouched late-2017 window it reached 28.9% precision and 34.0% recall, beating the purchase-time queue's precision by 7.3 points (interval 1.4 to 13.6).

Recommendation. - Do not staff a large purchase-time review queue. - If the purchase-time ranking is used, base it on recent route history (60 days), keep it to the top of each day's list, and size it to the current late rate. - Put human follow-up at the shipping-limit checkpoint. - Review how promise dates are set for short local routes. - Validate any intervention with a randomized pilot before claiming that it prevents delays.

---

1. Problem and decision

1.1 Context

Olist is a Brazilian marketplace that connects small sellers to large e-commerce sites. Each seller packs and hands the order to a carrier, mostly the national postal service, which then delivers it. At checkout the customer sees a promised delivery date. If that date is missed, the customer blames the marketplace.

1.2 Why it matters

Figure 1B shows the cost of a missed promise. On-time orders average a 4.29 review score with 6.6% one-star reviews. Late orders average 2.27, with 52.5% one-star reviews, which is 7.9 times the rate. Even though only 6.8% of delivered orders are late, they dominate the negative feedback a marketplace receives.

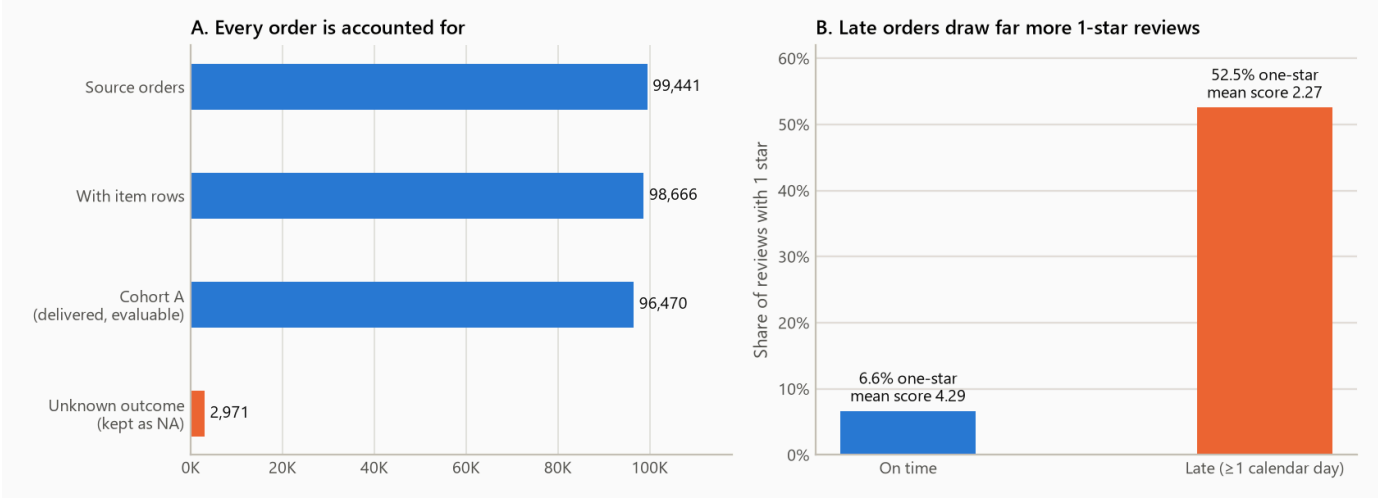


Figure 1. A: every source order is classified into the evaluable delivered cohort (Cohort A) or the unknown-outcome group. B: share of one-star reviews by delivery outcome. Reviews are used only as motivation, never as model inputs.

1.3 The decision we model

An operations team cannot inspect every order, so the decision is a ranking problem with a budget:

- **When:** once a day, at the end of the purchase day.
- **Who is eligible:** the orders placed that day.
- **How many:** at most `floor(0.10 × N)`, where N is the number of eligible orders that day. On a 205-order day that is 20 orders. Unused capacity is not carried over.
- **What may be used:** only information available when the order was placed: the basket, prices and freight, the promised date, the customer and seller locations, and the history of earlier orders that had already been delivered by then.

We use a daily quota rather than a probability threshold because the late rate swings widely from month to month (Figure 2). A threshold would produce wildly varying workloads, while a quota keeps the team's daily load fixed.

1.4 Questions

- **Q1.** How many late orders can such a queue catch, and at what false-alarm cost?
- **Q2.** Do machine-learning models beat simple, explainable rules?
- **Q3.** Is acting at purchase time worth it, or would a later checkpoint be better?

1.5 How this differs from common Olist analyses

Public notebooks on this dataset usually split orders at random, use delivery timestamps as features, and report accuracy. Each choice inflates results: random splits mix future months into training, delivery timestamps are unknown when the order is placed, and accuracy is meaningless when about 93% of orders are on time. We instead evaluate in time order, compute history features only from orders already delivered, keep orders with unknown outcomes visible, and report results in units an operations team can use: orders reviewed, delays caught and false alarms.

---

2. Data

2.1 Source

We use the public Brazilian E-Commerce Public Dataset by Olist (Kaggle), which has 9 linked tables and 99,441 orders purchased between 2016-09-04 and 2018-10-17. Every run checks the SHA-256 digest of each raw file. Customer and seller identities are anonymized, and locations are given as 5-digit postal prefixes.

2.2 Who is in the analysis

Figure 1A and the table below account for every order.

Group	Orders	How it is used
All source orders	99,441	Scored in the operational view (“Path B”)
Orders with at least one item	98,666	775 orders have no item rows
Cohort A: delivered with valid dates	**96,470**	Labelled; used for training and accuracy metrics
Unknown outcome	2,971	Label kept as missing, never set to “on time”

The 2,971 unknown-outcome orders are 1,107 shipped but never confirmed delivered, 625 canceled, 609 unavailable, 314 invoiced, 301 processing, 8 marked delivered without a delivery date, 5 created and 2 approved. Treating them as on time would credit the model for orders that never arrived. We therefore report precision with and without them (Section 4.2).

2.3 Label

Promised dates are stored at midnight, so we define **late** as delivered on a calendar day after the promised date. By this definition, 6,534 of 96,470 orders (6.77%) were late. A stricter clock-time definition gives 7,826 late orders (8.11%). The 1,292 orders in between arrived on the promised day itself, which a customer would consider on time. We re-run the main result with the strict label as a check.

2.4 Exploration: the late rate is not stable

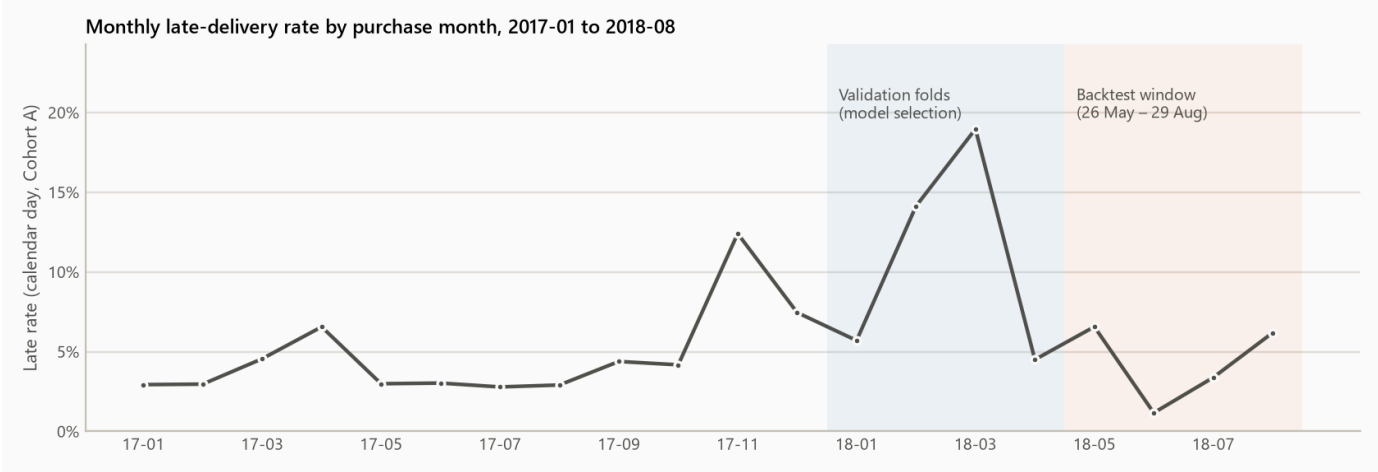


Figure 2. Share of Cohort A orders delivered late, by purchase month. Shaded areas are the validation months used to choose the model and the later backtest window.

The late rate averaged 5.6% in 2017. It jumped to 12.4% in November 2017, the Black Friday month and the busiest in the data (7,288 delivered orders), reached 14.1% in February and 19.0% in March 2018, fell to 1.2% in June 2018 and climbed back to 6.2% in August. Any model trained on one period is therefore tested on a different world. This drift turns out to be the central difficulty of the project.

2.5 Exploration: right-censoring at the model-selection date

A model can only learn from orders whose outcome is already known. At the model-selection cutoff (26 May 2018), 104 April orders had not yet been delivered, and 89 of them eventually arrived late. April’s late rate therefore looked like 3.24% at selection time but was really 4.50%. The orders that take longest to resolve are

the late ones, so recent periods always look better than they are. We keep this rule (no peeking at outcomes that were not yet known) and report its effect rather than hide it.

2.6 Data quality

There are 166 orders with a carrier hand-off time earlier than the purchase time and 23 delivered before the hand-off. Weights and dimensions are missing for 16 delivered orders. The geolocation table has 261,831 exact duplicate rows and 31 coordinates outside Brazil, which we drop before computing distances. None of these anomalies affects the label, and the models impute missing inputs.

---

3. Method

3.1 Time-ordered evaluation

Stage	Purchases evaluated	Training data	Purpose
Validation fold 1	January 2018	41,197 earlier orders already delivered	Choose the method
Validation fold 2	February 2018	47,794	Choose the method
Validation fold 3	March 2018	53,644	Choose the method
Validation fold 4	April 2018	60,468	Choose the method
Backtest	26 May to 29 August 2018 (96 days)	75,128, of which 7.11% late	Test the chosen method once

The training set at each cutoff contains only orders that had been delivered by that cutoff. For example, 2,048 orders still in transit at the backtest cutoff were left out. The backtest scores 19,661 orders, of which 19,294 are in Cohort A, and 674 of those (3.49%) were late. The window starts on 26 May, so May contributes only 691 orders.

3.2 Information available at purchase

The base feature set has 74 inputs (68 numeric, 6 categorical): basket size, price, freight, weight and volume; promised lead time; purchase calendar; customer and seller location, distance and whether they are in the same state; and history features. Outcome-history features (seller, route and state late rates) use only earlier orders that had already been delivered when the current order was placed, and volume features (recent seller orders) count only earlier purchases. An extended set adds 7 payment features. Payment records carry no timestamp, so their availability at checkout is an assumption, and we test the extended set separately.

3.3 Candidates

Family	Count	Settings
Rules	5	random order; constant prior; shortest promise first; seller's historical late rate; route history × promise tightness
Logistic regression	8	C ∈ {0.1, 1.0} × class weight {none, balanced} × feature set {base, extended}
Shallow decision tree	16	depth ∈ {3, 5} × min leaf ∈ {150, 300} × class weight × feature set

The models' probabilities are calibrated on the most recent 20% of each training window (Platt scaling).

The route-promise rule (`rule_region_lead`; the identifier is historical) scores an order as

> score = route late rate × promise tightness.

- **Route late rate:** the share of earlier delivered orders on the same seller-state to customer-state route that were late, shrunk toward the overall rate with a weight of 5 orders.
- **Promise tightness:** the median promised lead time in the training window (24 days) divided by this order's promised lead time. The order's lead time is first limited to between 3 and 90 days, and the ratio is limited to between 0.5 and 2.

A route with a bad record and a short promise therefore ranks first. The rule has two inputs and two fitted numbers: the median lead time and the overall late rate (7.11%).

3.4 Selection rule (fixed before any run)

We pick the candidate with the highest **mean daily top-10% recall** across the four validation folds, which is the share of late orders that would land in the daily queue. A model is preferred to a rule only if it beats the best rule. The candidate grid and the selection rule were written into the configuration before the first run.

3.5 Evaluation

- **Operational metrics:** alerts, late orders caught (true positives), false alarms, precision (share of alerts that were late), recall (share of late orders caught) and lift (precision divided by the base rate), all under the daily quota.
- **Ranking metrics:** average precision (the area under the precision-recall curve) and ROC-AUC.
- **Uncertainty:** a weekly block bootstrap. It resamples whole calendar weeks (15 in the backtest) 2,000 times, which keeps the strong week-to-week correlation in delays.
- **Unknown outcomes:** precision is reported as a range between treating unknown-outcome alerts as false alarms and treating them as genuine problems.
- **Cost:** three cost scenarios fixed in the configuration (Section 4.7).

Two analyses were designed after we had seen backtest results: the shipping-limit signal (Section 4.6) and the comparison with the shortest-promise rule (Section 4.4). They are labelled post hoc wherever they appear.

3.6 A second, pre-registered design and test period

The 2018 results suggested specific weaknesses, so we wrote down improvements, froze them in new configuration files, and only then ran them.

- **Censoring-aware labels.** An order that has passed its promised date undelivered is already known to be late, so it can enter training and selection from the next day instead of waiting for delivery. This recovers the late orders that the delivery-only rule throws away. On the 2018 contract it adds 652 training orders to the April fold and 107 to the final training set, and at selection time it raises the April late count from 217 to 278.
- **Three new rules.**
  - The route-promise rule with route history limited to the last 60 days.
  - The same rule with a censoring-aware 60-day route rate.
  - A promise-slack rule: the promised lead time minus the route's recent actual delivery time.
- **A new feature version for the models** (`base_v3`), which adds these three route measures.
- **An explicit selection margin.** A model must beat the best rule by at least 0.5 percentage points.

The design was tested twice:

- **Late 2017 (the real test).** Monthly validation folds from June to September 2017, selection on 26 October 2017, and a backtest of every order placed from 26 October to 31 December 2017 (67 days, including Black Friday): 14,023 orders, 13,579 delivered, 1,358 late (10.0%). The configurations were frozen at 12:40 on 20 October 2026, before any model had been scored on this period (Appendix A).
- **The 2018 window again.** This run is post hoc, because that window had already been inspected.

The fixed pipeline code also re-ran the original v2 design as `experiment-v2-002`, with 2,000 bootstrap draws as configured (the first run used 400). Its predictions are identical, byte for byte, to the original run.

---

4. Results

4.1 Model selection: a rule beats the models

Candidate	Validation recall (mean of 4 folds)	Worst fold	Validation avg. precision	Backtest recall	Backtest avg. precision	Backtest ROC-AUC
Route-promise rule (selected)	**25.2%**	23.2%	0.206	**17.4%**	0.0604	0.6416
Best logistic regression (C = 0.1, extended)	21.0%	2.9%	0.168	15.1%	0.0516	0.615
Best tree on validation (depth 5, leaf 300)	15.6%	12.6%	0.135	12.3%	0.0380	0.509
Seller history rule	14.0%	10.8%	0.121	12.0%	0.0349	0.491
Constant prior	9.0%	8.4%	0.105	11.1%	0.0349	0.500
Random order	8.9%	7.2%	0.103	10.4%	0.0357	0.510
Shortest promise first	4.9%	2.3%	0.097	**22.4%**	**0.1203**	**0.7503**

Recall is the share of late orders caught by the daily top-10% queue. Appendix B lists all 29 candidates.

The route-promise rule led validation by 4.2 percentage points and was the most stable candidate. Its weakest fold still caught 23.2% of late orders. The best logistic regression was erratic: it caught 41.5% in April but only 2.9% in February. Payment features made no material difference. Shallow trees were weaker still. In the backtest, none of the 24 models caught more late orders than the rule. The best of them reached 15.3%.

Why does a two-input rule beat 74-feature models? Most of the predictive information sits in two things: where the order is going and how much slack the promise leaves. The rule encodes both directly and adapts automatically as route histories update. The models had to learn the same structure from noisy, shifting data. A random ranking catches about 10% of late orders by construction, so every model added only a little on top of that.

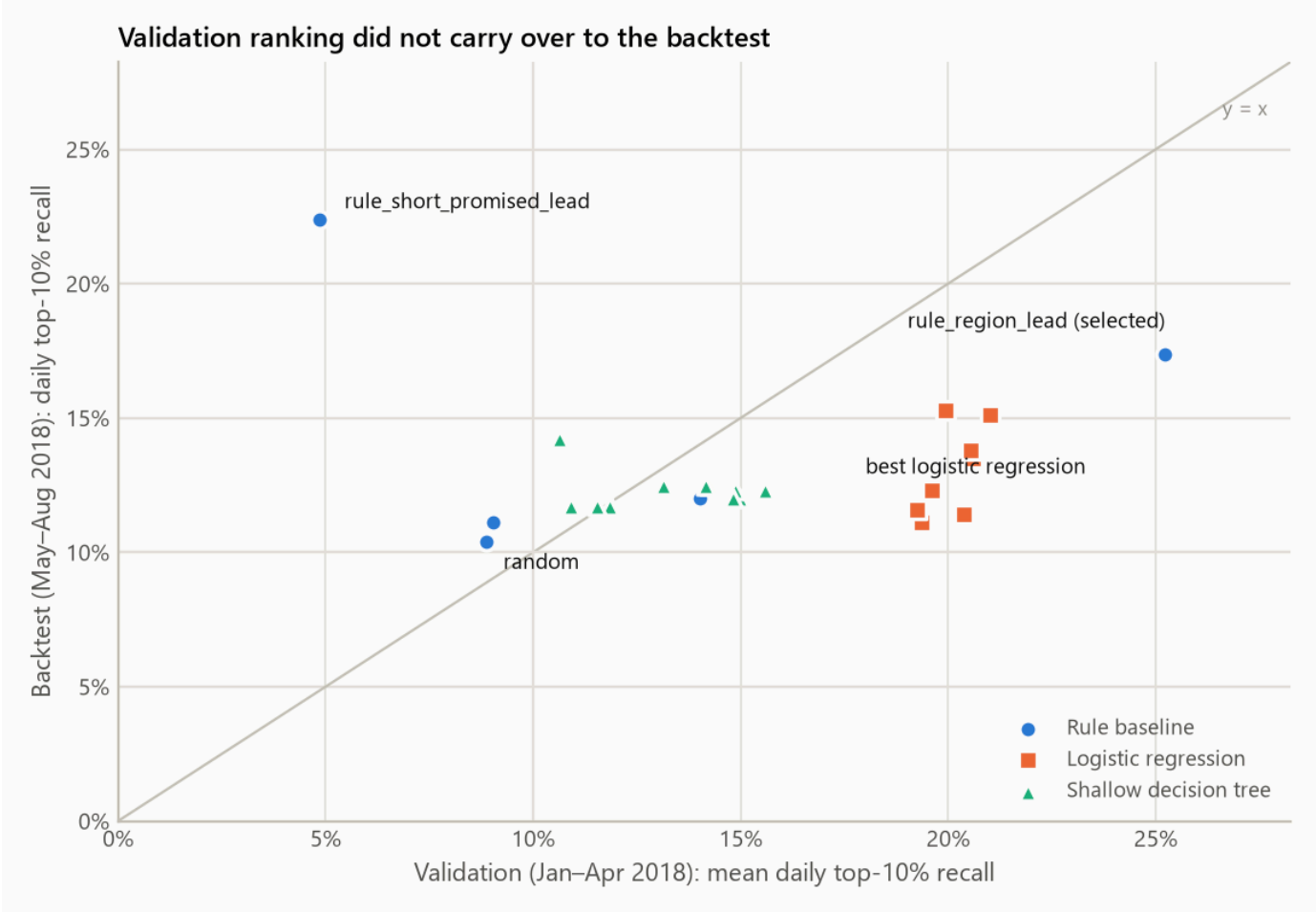


Figure 3. Each point is one candidate. If validation predicted the backtest well, points would lie near the diagonal. They do not.

4.2 How well the selected rule works in the backtest

Measure	Value	95% interval (weekly block bootstrap, 2,000 draws)
Alerts (daily quota, Cohort A)	1,885	—
Late orders caught	117 of 674	—
False alarms	1,768	—
Recall	17.4%	12.7% to 27.0%
Precision	6.2%	4.2% to 8.4%
Lift over the 3.49% base rate	1.78	—
Average precision	0.0604	0.0464 to 0.0805
ROC-AUC	0.6416	0.586 to 0.706 (analysis layer)

The rule is clearly better than chance: the whole precision interval lies above the base rate. But it is weak. The team would review 16.1 orders for each late order found, of which 15.1 would be false alarms.

- **Strict clock-time label.** The same queue catches 150 late orders, with 8.0% precision and 14.7% recall, so the conclusion does not depend on the label definition.

- **Operational view (Path B).** Scoring all 19,661 orders gives 1,922 alerts, about 20 a day. Of these, 45 (2.3%) are orders whose outcome is unknown: 35 shipped but never confirmed delivered, 7 canceled, 2 invoiced and 1 processing. True precision therefore lies between 6.0% (all 45 are false alarms) and 8.4% (all 45 are genuine problems).

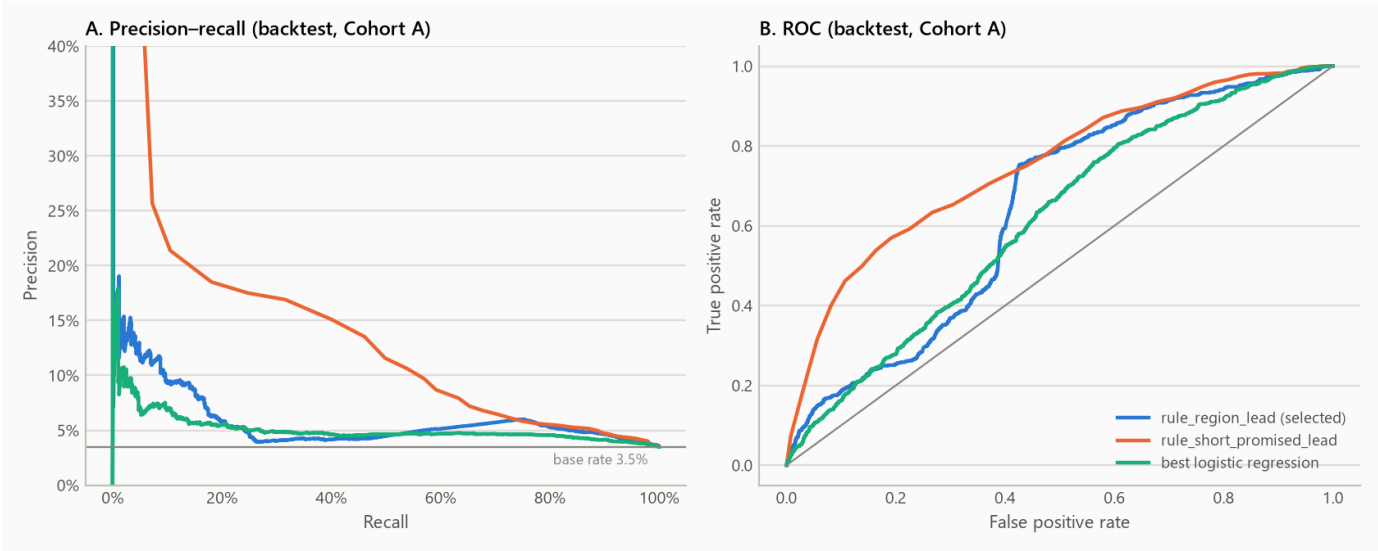


Figure 4. Backtest ranking curves for the selected rule, the shortest-promise rule and the best logistic regression.

4.3 Capacity trade-off

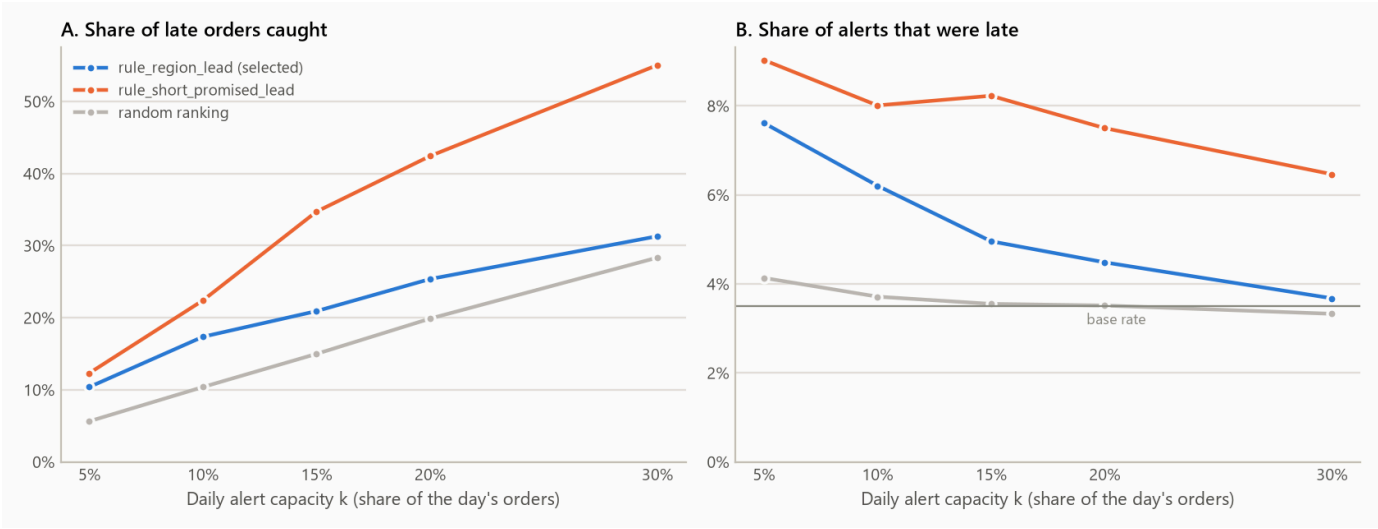


Figure 5. Recall and precision as the daily review budget changes, under the same daily quota rule.

Daily budget	Alerts	Late caught	Precision	Recall	Lift
5%	919	70	7.6%	10.4%	2.18
10%	**1,885**	**117**	**6.2%**	**17.4%**	**1.78**
15%	2,844	141	5.0%	20.9%	1.42
20%	3,813	171	4.5%	25.4%	1.28
30%	5,739	211	3.7%	31.3%	1.05

Precision falls steadily as the budget grows. At a 30% budget the queue is barely better than random (lift 1.05). The rule's value is concentrated at the very top of each day's list, so a larger team would mostly review on-time orders.

4.4 What changed between the two periods

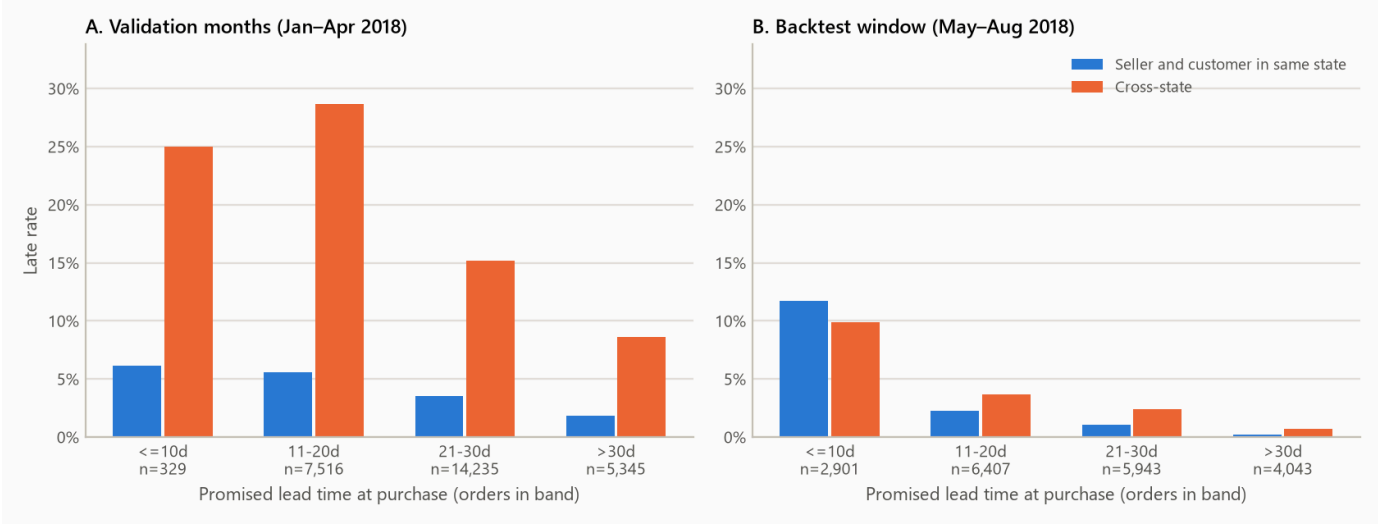


Figure 6. Late rate by promised lead time and whether seller and customer are in the same state, for the validation months (A) and the backtest window (B). The n under each band is the number of orders in it.

Validation months (January to April 2018). - 10.8% of orders were late. - Cross-state orders were the problem: 14.2% late against 4.9% within a state, and 83.7% of all late orders crossed a state line. - Rio de Janeiro customers saw 22.0% of orders arrive late. - Promises of 10 days or less were rare (329 orders, 1.2% of the total).

Backtest window (26 May to 29 August 2018). - Only 3.5% of orders were late. - Same-state orders were now riskier: 5.1% late against 2.4% for cross-state, and 59.3% of late orders stayed within one state. - Promises of 10 days or less had become common: 2,901 orders, 15.0% of the total, 90.6% of them local. They were late 11.6% of the time and accounted for 49.9% of all late orders. - São Paulo to São Paulo orders with such short promises (2,410 orders) produced 279 late deliveries, 41.4% of the window's total. - Promises longer than 30 days were almost never missed (0.7%).

This pattern is consistent with a change in how promise dates were set for local orders. Short promises appeared in large numbers and were often missed. We cannot verify the cause from the data, but it explains the ranking flip in Figure 3. The route-promise rule leans on route history, which was built mostly on the earlier cross-state problem, so it lagged. The shortest-promise rule was useless in validation, where short promises were rare and safe. In the backtest it was the best candidate, with 22.4% recall.

Should we have picked the shortest-promise rule? No, for two reasons:

1. **Its advantage is not statistically clear.** A paired weekly block bootstrap of the difference in recall gives an interval from −13.4 pp to +13.9 pp, with 74.3% of draws favouring it (post hoc).
2. **Switching after looking at the test would turn the backtest into a training set,** and the reported performance would no longer be an honest estimate.

The lesson is different: one backtest window covers a single regime, so performance must be monitored and the method re-selected on recent data.

4.5 The scores are a ranking, not a probability

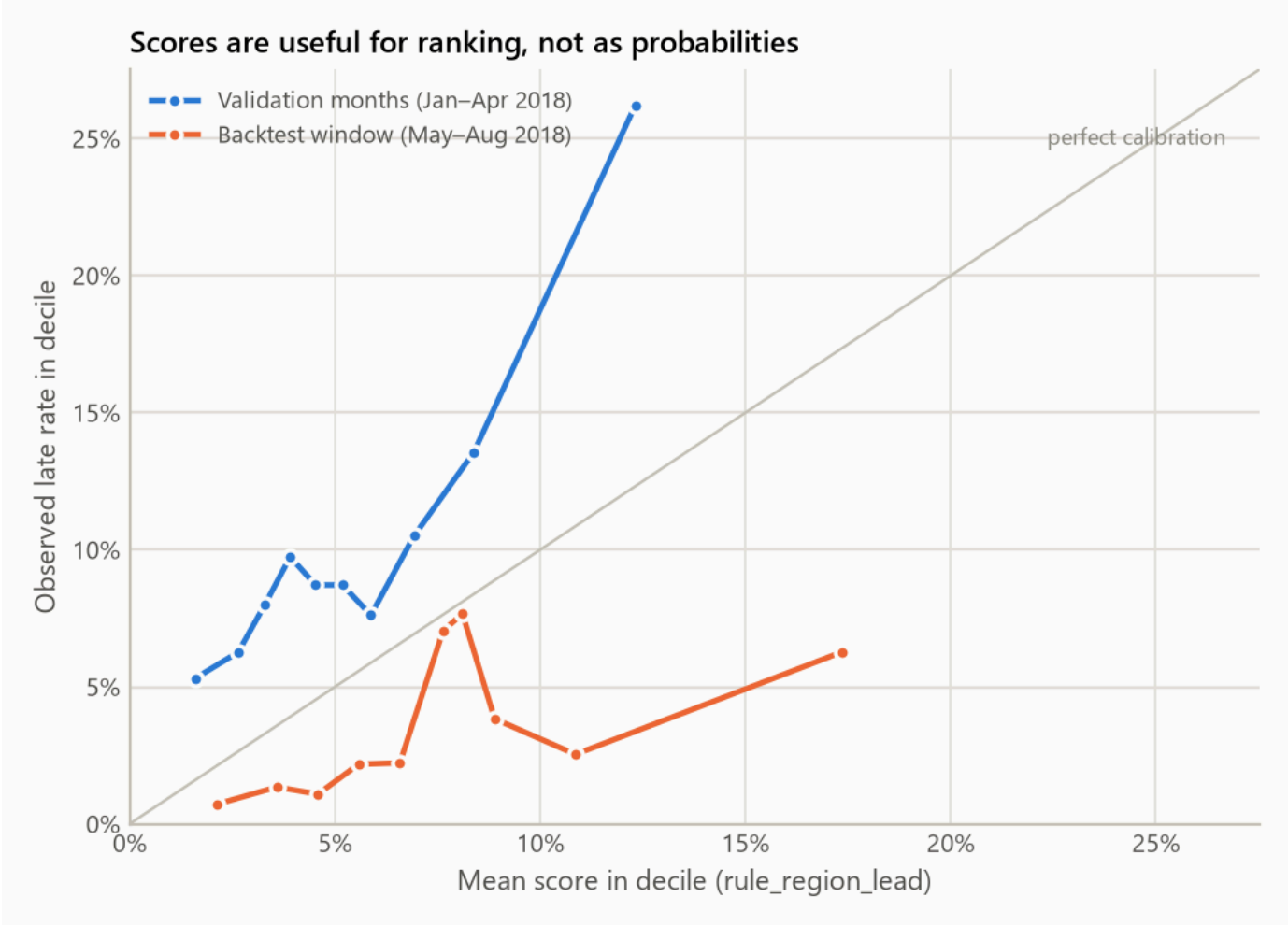


Figure 7. Observed late rate against the rule's score, by score decile, in each period.

In the validation months the rule's scores averaged 5.5% while 10.5% of orders were late, so it under-predicted by about half. In the backtest the scores averaged 7.5% against 3.5% observed, an over-prediction by a factor of 2.16. In the backtest the top decile scored 17.3% but only 6.3% of those orders were late. The scores are useful for ordering orders within a day, but they should never be read as probabilities or compared with a fixed threshold. This is why the daily quota matters: it uses only the order of the scores.

4.6 A later signal: the seller misses the shipping-limit date (post hoc)

Where the extra time goes. Figure 8 looks backwards at completed orders (this information is never used for prediction). Late orders took 22.9 more days than on-time ones. - Carrier transit accounts for 19.9 of those days (87%): 27.9 days against 8.0. - The seller hand-off accounts for 2.9 days (13%): 5.5 days against 2.6. - 27.8% of late orders, against 7.7% of on-time orders, were handed to the carrier after the seller's shipping-limit date, a 3.6-fold difference.

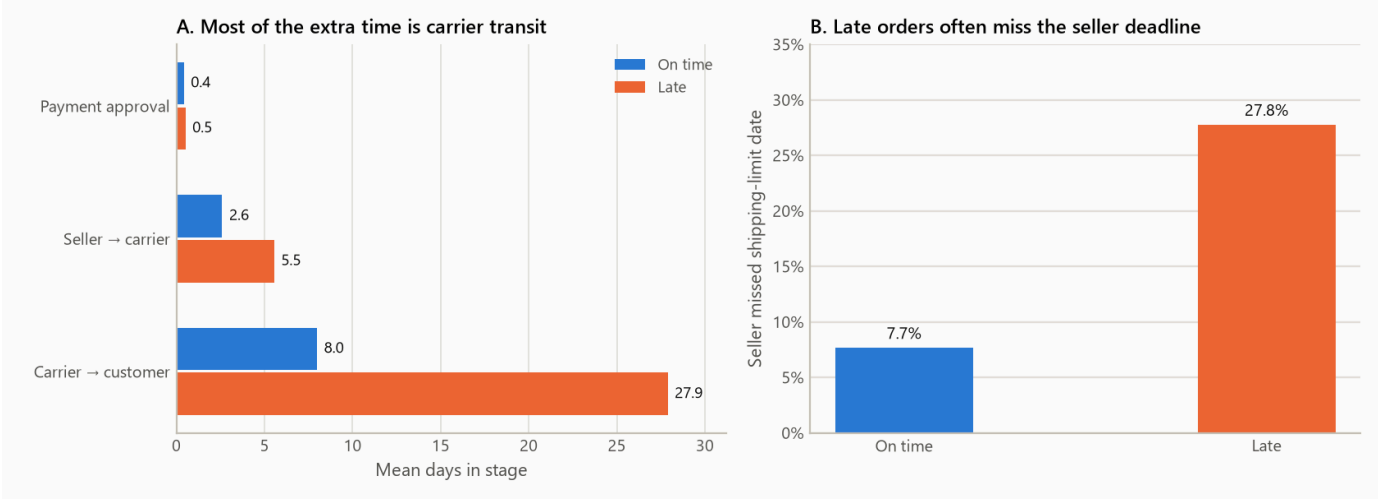


Figure 8. A: mean days in each fulfilment stage. B: share of orders that the seller handed to the carrier after the shipping-limit date. Ex post, all delivered orders.

Turning this into a policy. That suggested a second checkpoint. At each order's shipping-limit deadline, flag it if it has not yet been handed to the carrier. This is known at the deadline, needs no model and has no fitted parameters. We evaluated it on the same backtest orders:

Policy	Alerts	Late caught	Precision	Recall	Lift
Purchase-time rule, daily top 10%	1,885	117	6.2%	17.4%	1.78
Seller missed shipping limit	**1,575**	**306**	**19.4%**	**45.4%**	**5.6**
Either signal	3,315	388	11.7%	57.6%	—

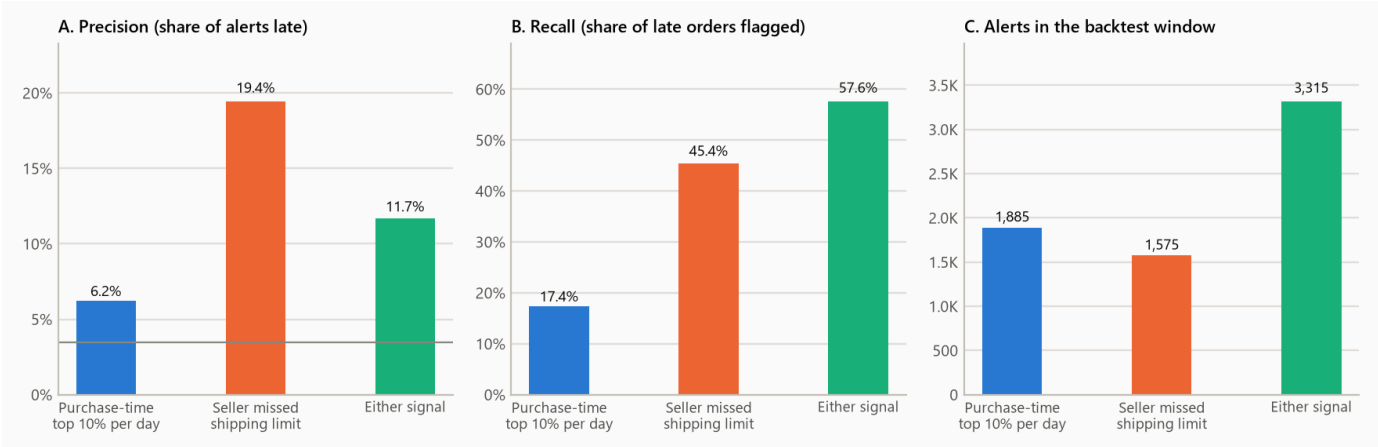


Figure 9. Backtest precision, recall and alert volume of the two signals and their union (post hoc).

- **Sharper.** The shipping-limit signal raised 1,575 alerts (8.2% of orders, about 16 a day) and reached 3.1 times the precision and 2.6 times the recall of the purchase-time queue. The precision gain is 13.2 percentage points, with a bootstrap interval of 4.3 to 20.1 points.
- **Fair comparison.** At the same alert volume, the purchase-time ranking would reach only 7.2% precision.
- **The signals are complementary.** Only 35 late orders were flagged by both.

- **There is still time to act.** When flagged, the median order had 13.6 days left before its promised date. For orders that ended late the median was 3.2 days, and only 2.2% were flagged after the promise had already passed.
- **It depends on the regime.** In the validation months the same signal had 22.8% precision but only 19.4% recall, because early-2018 delays came from carrier transit, which this signal cannot see.

4.7 Is it worth acting? Cost scenarios fixed in advance

The configuration defines three hypothetical scenarios, in arbitrary money units (the configuration labels them in dollars). For an alert with precision p, an action pays off when p × benefit exceeds the contact cost plus (1 − p) × the false-alarm cost.

Scenario	Benefit per caught delay	Contact cost	False-alarm cost	Break-even precision	Purchase-time queue, best budget	Seller missed shipping limit
Conservative	2.00	0.30	0.20	22.7%	Loses money at every budget; −35.5 per 1,000 orders at 10%	−5.9
Moderate	5.00	0.25	0.15	7.8%	Loses money; −0.4 per 1,000 at 5%, −7.8 at 10%	**+49.0**
High SLA penalty	10.00	0.20	0.10	3.0%	**+31.9** per 1,000 at 10% (best)	**+135.7**

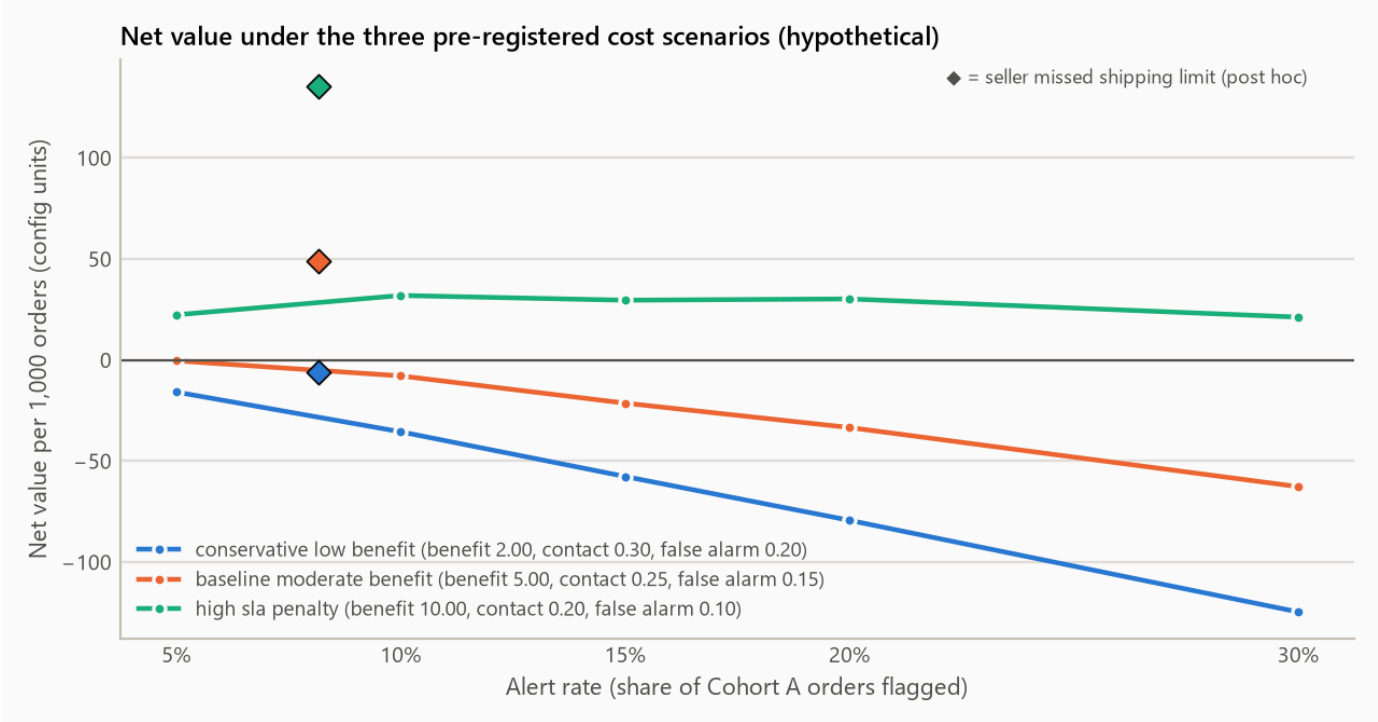


Figure 10. Hypothetical net value per 1,000 delivered orders as a function of the alert rate. Lines: purchase-time queue. Diamonds: shipping-limit signal (post hoc).

A purchase-time queue at 6.2% precision is worth running only when a caught delay is worth at least about 16 times the cost of a contact. That is the case only in the most optimistic scenario. The shipping-limit signal clears the moderate scenario comfortably. If an avoided delay also saves the separately configured missed-delay cost, the break-even precisions fall to 15.6%, 5.6% and 2.0%, which changes no conclusion. The scenarios are illustrative, and the value of catching a delay depends entirely on whether the action actually prevents it, which observational data cannot show.

4.8 Second, pre-registered test: late 2017

Candidate (late-2017 test)	Validation recall (Jun–Sep 2017)	Worst fold	Backtest recall	Backtest precision	Backtest avg. precision
60-day route × promise (selected)	**19.1%**	16.4%	**21.1%**	**21.6%**	0.151
60-day censoring-aware route × promise	17.6%	13.1%	23.7%	24.3%	0.153
Route × promise (the v2 winner)	18.1%	12.3%	13.4%	13.7%	0.119
Best model (tree, depth 5, leaf 300)	18.7%	6.3%	17.3%	17.7%	0.122
Promise slack	16.0%	12.3%	8.3%	8.5%	0.123
Shortest promise first	12.7%	9.7%	4.2%	4.3%	0.092
Random order	10.3%	9.7%	10.2%	10.5%	0.102
Seller missed shipping limit (no ranking)	—	—	34.0%	28.9%	—

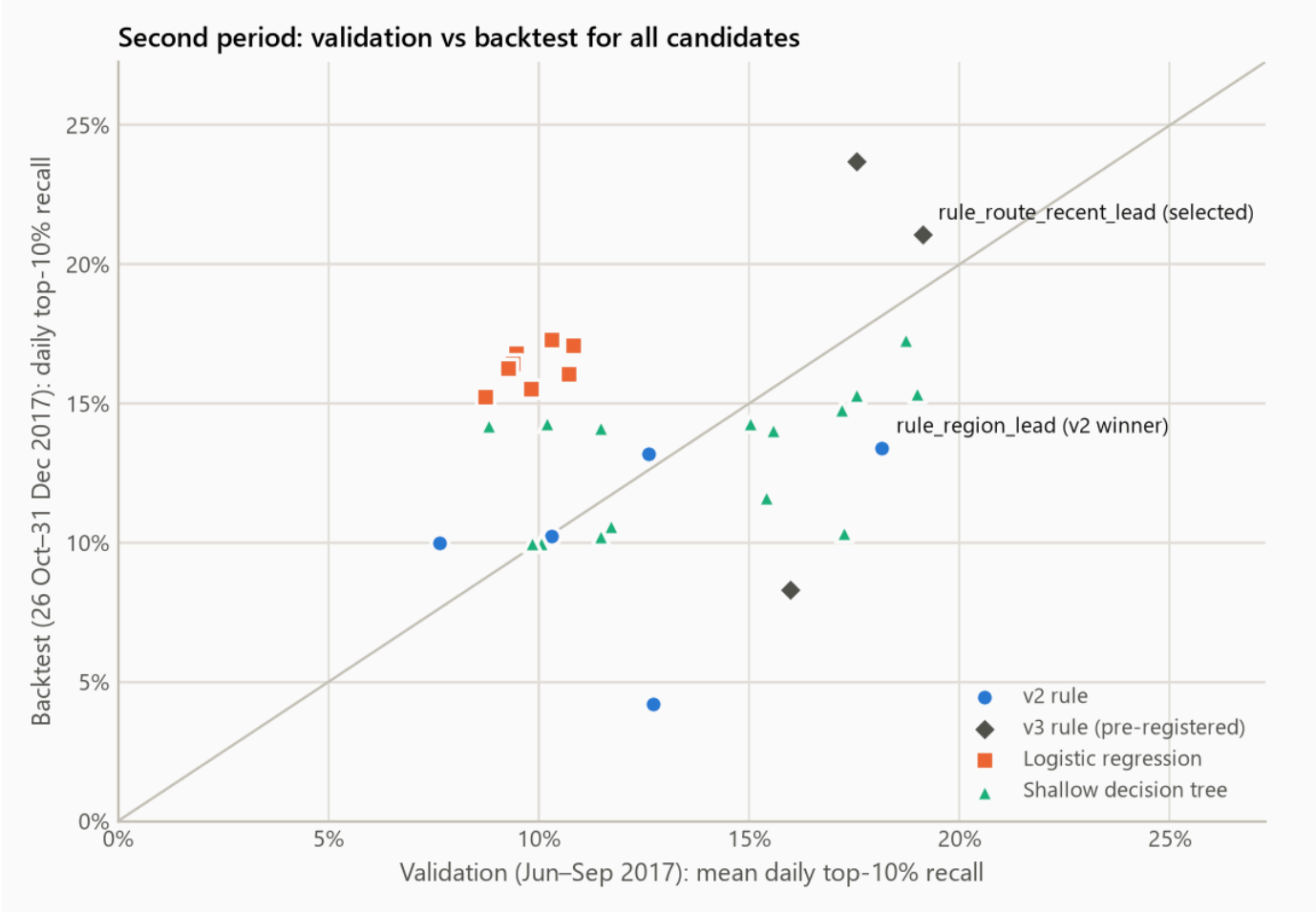


Figure 11. Each point is one of the 32 candidates in the pre-registered design, selected on June to September 2017 and tested on 26 October to 31 December 2017.

What changed. - This period was very different from 2018. Late rates in the validation months were low (2.8% to 4.4%), and the backtest window contained the November 2017 surge: 10.0% of delivered orders were late. - The selection rule again picked a rule. The best model was 0.4 percentage points behind on validation, short of the 0.5-point margin, and its worst validation fold caught only 6.3% of late orders.

Results on the unseen window. - **Recency pays.** The selected 60-day route rule caught 21.1% of late orders, at 21.6% precision (lift 2.16). The original route rule caught 13.4%. The paired weekly block bootstrap puts the gain at 4.5 pp to 12.8 pp, and all 2,000 draws favour the 60-day rule. The selected rule's own 95% interval is 17.8% to 25.9%. - **Censoring-aware history helps a little more.** The censoring-aware variant reached 23.7%, the best of all 32 candidates, but it ranked only fifth in validation, so selection did not pick it. - **The 2018 hindsight winners do not generalise.** Promise slack (8.3%) and shortest promise (4.2%) were worse than a random queue (10.2%). Choosing either after looking at the 2018 backtest would have been a mistake, which supports keeping the pre-registered choice there (Section 4.4). - **The shipping-limit signal holds on fresh data.** It raised 1,599 alerts (11.8% of delivered orders, about 24 a day), with 28.9% precision and 34.0% recall. Its precision beats the purchase-time queue by 7.3 points (paired interval 1.4 to 13.6, 99% of draws positive). Orders that ended late still had a median of 17.3 days before their promise when flagged. Its advantage is smaller than in 2018 because purchase-time ranking is easier when many orders are late.

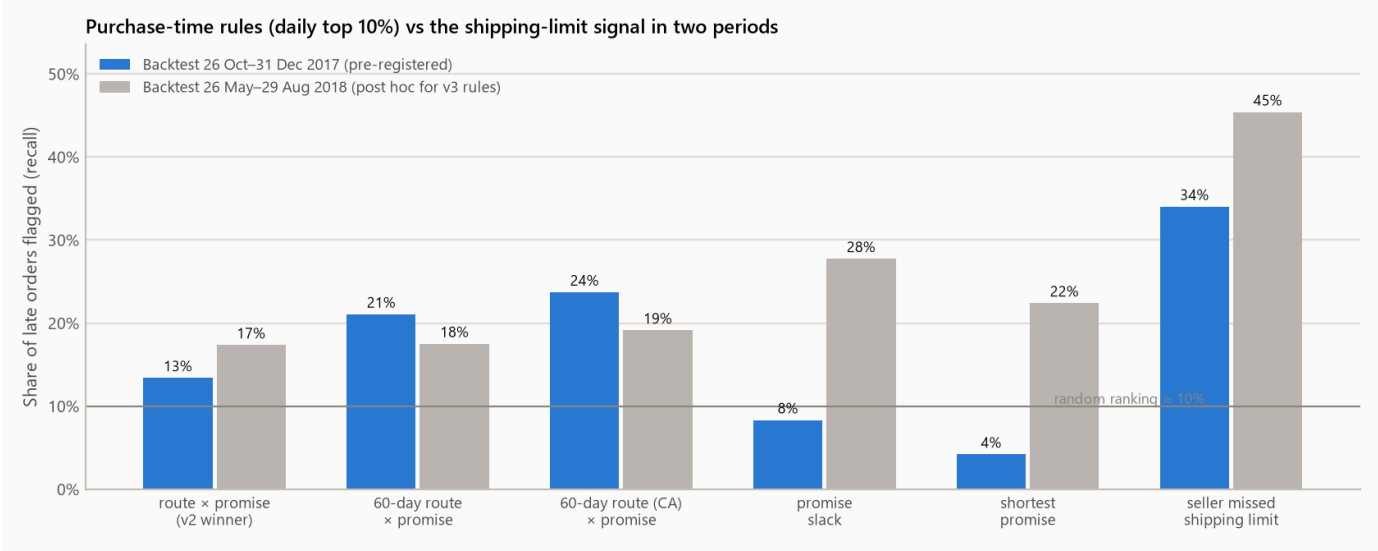


Figure 12. Recall of the daily top-10% queue for each rule, and of the shipping-limit flag, in the late-2017 backtest (pre-registered) and the 2018 backtest (post hoc for the v3 rules). The shipping-limit flag is not a ranked queue; it flagged 11.8% of orders in 2017 and 8.2% in 2018.

Costs in a high-delay period. At 21.6% precision, the purchase-time queue now clears the moderate scenario's 7.8% break-even and earns +69.4 per 1,000 delivered orders. In the conservative scenario it is roughly break-even (−2.5). The shipping-limit signal earns +128.1 in the moderate scenario and +16.0 even in the conservative one. The value of acting at purchase time therefore depends strongly on the current late rate.

4.9 The new design on the 2018 window (post hoc)

On the already-inspected 2018 contract, the v3 design selected the original route rule again: censoring-aware labels did not change the choice. In hindsight, the 60-day rules would have caught 17.5% (plain) and 19.1% (censoring-aware) of late orders, almost the same as the selected rule's 17.4%. Promise slack would have caught 27.7%, but it ranked only seventh in validation, and it failed in late 2017. The lesson from both windows: recency-based route history is a robust improvement, and rules that shine in one backtest without validation support should not be trusted.

---

5. Recommendations

1. **Do not build a large human review queue at purchase time.** In the low-delay 2018 window it needs about 16 reviews per late order found and pays only under optimistic costs. In the high-delay late-2017 window it pays under the moderate scenario, so any purchase-time effort should scale with the current late rate.
2. **If a purchase-time ranking is used, base it on recent route history.** The 60-day route rule won the pre-registered test and beat the original rule by 7.7 points of recall. Keep the queue to the top of each day's list (precision is highest there) and prefer cheap automated actions such as an expectation-setting message or a seller reminder.
3. **Put human effort at the shipping-limit checkpoint.** Flag orders not handed to the carrier by the deadline. It was sharper than the purchase-time queue in both periods: 19.4% vs 6.2% precision in 2018, and 28.9% vs 21.6% on the untouched late-2017 window. Late orders still had a median of 3.2 to 17.3 days before the promise when flagged. Run it first as a randomized pilot.

4. **Review promise-setting for short local routes.** São Paulo to São Paulo orders promised in 10 days or less produced 41% of the 2018 backtest's late deliveries. Adding a buffer there may prevent more delays than any warning system.
5. **Monitor and re-select.** Each month, track the late rate, the share of short promises, the score-to-outcome ratio and recall on recently matured orders. Re-run selection on recent folds, and promote the censoring-aware 60-day rule if it keeps leading.
6. **Measure the effect of interventions.** Catching a delay early does not prove it can be prevented. A randomized pilot (alert versus no alert) is the only way to estimate the benefit that the cost scenarios assume.

---

6. Critical evaluation

6.1 Limitations of the data

- **One marketplace, one era.** The data cover a single marketplace in 2016–2018. Carrier networks, promise algorithms and seller mix may have changed since.
- **Promise dates are set by Olist itself.** The label measures whether Olist's own estimate was met, not delivery speed. A change in the estimator changes the label (Section 4.4), and the data do not tell us when or why the estimator changed.
- **Timestamp quality.** Hand-off and delivery times may be recorded late or inconsistently, as the 166 hand-offs before purchase show. The shipping-limit signal assumes hand-offs are recorded promptly.
- **Unknown outcomes.** 2,971 orders have no observable outcome. We bound their effect on precision but cannot resolve it.
- **Unknown availability.** Payment features have no timestamp, so their availability at checkout is assumed. They did not help, which limits the damage.

6.2 Limitations of the method

- **Unrepresentative folds.** Each design was selected on four months that differed from its test window: the 2018 folds contained the worst disruption in the data, and the 2017 folds were calm before the November surge.
- **Short test windows.** 96 days (15 weeks) in 2018 and 67 days (10 weeks) in late 2017 give wide intervals. For example, 2018 recall could plausibly be anywhere from 12.7% to 27.0%.
- **The 2018 backtest is not a blind test.** During the project audit we computed descriptive statistics of this window. Shortly before the first formal run, two smoke runs (with a reduced grid) also scored it, and pipeline code was edited in between. The configuration, candidate grid and selection rule were fixed earlier and were not changed, and a later re-run on the final code reproduced the smoke predictions exactly. Still, it is best described as a diagnostic holdout.
- **The second test is fresh but earlier in time.** No model had been scored on late 2017 when the v3 design was frozen, but the v3 ideas came from studying 2018, and late 2017 comes before 2018. It tests whether the ideas transfer to a different period, not whether they predict the future. Monthly late rates for 2017 had been plotted beforehand, so the analysts knew a November surge existed.
- **Bootstrap replicates.** The first formal run (`experiment-v2-001`) used 400 bootstrap draws instead of the configured 2,000 because the number was hard-coded. The fixed code re-ran it with 2,000 draws as `experiment-v2-002`, with identical predictions.

- **Simplifications.** Orders with several sellers use the route of the main seller. Model probabilities were calibrated on an earlier period than the one they were applied to, which drift made obsolete.
- **Post hoc analyses.** The shipping-limit signal and the shortest-promise comparison were designed after seeing the 2018 results. The shipping-limit signal has since held on the untouched late-2017 window, but it remains associational.

6.3 Limitations of the conclusions

- All results are associations. Nothing here shows that contacting a seller or customer prevents a delay.
- The cost values are hypothetical. They show where the break-even lies but not what Olist would actually gain.
- "Machine learning did not help" holds for these feature sets, these simple models and these two periods. Richer real-time data, such as carrier scan events, could change the picture, which is consistent with the stronger signal found at the hand-off checkpoint.

---

7. Conclusion

At the moment an order is placed, delivery delays on Olist are only weakly predictable, and what predicts them shifts within months. Across two independent test periods, transparent route-and-promise rules matched or beat 24 machine-learning models. The second, pre-registered test showed that using only recent route history makes the rule clearly better (21.1% against 13.4% recall), and that rules which looked best in hindsight do not transfer. Whether a purchase-time queue is worth running depends on how many orders are currently late.

A missed shipping-limit deadline was the sharper warning in both periods, and the concentration of 2018 delays in short local promises points to fixing how promises are set. For the operations team, the decision is to move human attention to the hand-off checkpoint, keep any purchase-time effort small and based on recent history, and test interventions before scaling them.

Appendix A. Reproducibility and evidence

- **Formal run.** `artifacts/runs/experiment-v2-002`: VERIFIED, configuration `configs/experiment-v2.json`, raw data checked against SHA-256 digests. It is the fixed-code re-run of `experiment-v2-001`, with identical predictions and 2,000 bootstrap draws.
- **Second design.** `experiment-v3-late2017-001` (pre-registered test period) and `experiment-v3-2018-001` (post hoc), both VERIFIED. Configurations: `configs/research-v3.json`, `configs/experiment-v3-late2017.json`, `configs/experiment-v3.json`.
- **Analysis layers.**
  - `scripts/report_analysis.py` writes `artifacts/analysis/experiment-v2-002-report/`. It refuses to run unless the formal run's headline counts are reproduced exactly.
  - `scripts/robustness_analysis.py` writes `artifacts/analysis/robustness-report/` for the second design.

- **Document check.** `scripts/check_docs_consistency.py` confirms that every number in this report, the slides and the briefs matches a value in the two `report_facts.json` files or the configurations, and that every figure link resolves.
- **Exposure timeline (see `docs/test-exposure-log.md`).**
    1. On 2026-10-19 a legacy diagnostic run was interrupted and never scored the 2018 window.
    2. The v2 configurations were frozen at 00:37 on 20 October.
    3. Smoke runs scored the 2018 window at 02:30 and 02:37.
    4. The first formal run started at 02:47.
    5. The v3 configurations were frozen at 12:40.
    6. The late-2017 run, the first to score that period, started at 12:46.

Appendix B. All candidates

Candidate	Validation rank	Validation recall	Backtest recall	Backtest avg. precision	Backtest ROC-AUC
rule_region_lead	1	25.2%	17.4%	0.0604	0.642
lr_C0p1_cw_none__extended_purchase_assumptions_v2	2	21.0%	15.1%	0.0516	0.615
lr_C1p0_cw_none__extended_purchase_assumptions_v2	3	20.6%	13.5%	0.0490	0.607
lr_C1p0_cw_none__base_v2	4	20.6%	13.8%	0.0486	0.605
lr_C0p1_cw_balanced__extended_purchase_assumptions_v2	5	20.4%	11.4%	0.0465	0.602
lr_C0p1_cw_none__base_v2	6	19.9%	15.3%	0.0514	0.612
lr_C0p1_cw_balanced__base_v2	7	19.6%	12.3%	0.0464	0.599
lr_C1p0_cw_balanced__extended_purchase_assumptions_v2	8	19.4%	11.1%	0.0442	0.581
lr_C1p0_cw_balanced__base_v2	9	19.3%	11.6%	0.0440	0.580
dt_d5_leaf300_cw_none__base_v2	10	15.6%	12.3%	0.0380	0.509
dt_d5_leaf150_cw_none__extended_purchase_assumptions_v2	11	15.0%	12.0%	0.0364	0.508
dt_d5_leaf300_cw_none__extended_purchase_assumptions_v2	12	14.9%	12.3%	0.0380	0.509
dt_d5_leaf150_cw_none__base_v2	13	14.8%	12.0%	0.0372	0.509
dt_d3_leaf300_cw_none__base_v2	14	14.2%	12.5%	0.0374	0.509
dt_d3_leaf300_cw_none__extended_purchase_assumptions_v2	15	14.2%	12.5%	0.0374	0.509
dt_d3_leaf150_cw_none__base_v2	16	14.2%	12.5%	0.0374	0.509
rule_seller_history	17	14.0%	12.0%	0.0349	0.491
dt_d3_leaf150_cw_none__extended_purchase_assumptions_v2	18	13.1%	12.5%	0.0374	0.509
dt_d3_leaf300_cw_balanced__extended_purchase_assumptions_v2	19	11.9%	11.7%	0.0370	0.513
dt_d3_leaf300_cw_balanced__base_v2	20	11.9%	11.7%	0.0370	0.513
dt_d3_leaf150_cw_balanced__base_v2	21	11.9%	11.7%	0.0370	0.513
dt_d3_leaf150_cw_balanced__extended_purchase_assumptions_v2	22	11.9%	11.7%	0.0370	0.513
dt_d5_leaf300_cw_balanced__extended_purchase_assumptions_v2	23	11.6%	11.7%	0.0565	0.671
dt_d5_leaf300_cw_balanced__base_v2	24	10.9%	11.7%	0.0565	0.671
dt_d5_leaf150_cw_balanced__base_v2	25	10.6%	14.2%	0.0589	0.682
dt_d5_leaf150_cw_balanced__extended_purchase_assumptions_v2	26	10.6%	14.2%	0.0589	0.682
global_prior_baseline	27	9.0%	11.1%	0.0349	0.500
rule_random	28	8.9%	10.4%	0.0357	0.510
rule_short_promised_lead	29	4.9%	22.4%	0.1203	0.750

Source: `tables/leaderboard.csv` *in the analysis layer. Recall is daily top-10% recall; validation recall is the mean over the four folds.*

Appendix C. Definitions

- **Cohort A.** Delivered orders with at least one item and valid purchase, promise and delivery dates.
- **Late.** Delivered on a calendar day after the promised date.
- **Daily quota.** Each day, the `floor(0.10 × N)` highest-scored orders among that day's N orders. Ties are broken by earlier purchase time, then by order id.
- **Precision, recall, lift.** The share of alerts that were late; the share of late orders that were alerted; precision divided by the base late rate.
- **Average precision.** The precision averaged over all recall levels, which summarizes the precision-recall curve.
- **Weekly block bootstrap.** Resampling whole calendar weeks with replacement, so that within-week correlation is preserved.